

Sistem Monitoring Suhu *Storage* di Industri Berbasis Bahasa Pemrograman Rust

Disusun untuk Memenuhi Tugas Mata Kuliah:

Algoritma dan Pemrograman

Semester Genap Tahun Akademik 2025/2026

Dosen Pengampu:

Ahmad Radhy, S.Si., M.Si.



Disusun oleh:

Ibeth Novvalent Hizbu Syawalullah Santos 2042251063

Adzikra Maulana Fabiano Ramadhan 2042251069

Departemen Teknik Instrumentasi
Institut Teknologi Sepuluh Nopember
Surabaya
2026

Daftar Isi

1	Pendahuluan	3
1.1	Latar Belakang	3
1.2	Rumusan Masalah	3
1.3	Tujuan	3
1.4	Manfaat	4
2	Studi Kasus Instrumentasi	4
2.1	Deskripsi Sistem	4
2.2	Spesifikasi Sistem	4
2.3	Kategori Status Suhu	4
2.4	Hubungan dengan Teknik Instrumentasi	5
3	Algoritma dan Flowchart	5
3.1	Algoritma Sistem	5
3.2	Flowchart Sistem	6
4	Penjelasan Program Rust	8
4.1	Struktur Program	8
4.2	File <code>main.rs</code> — Pintu Masuk Program	8
4.3	File <code>sensor.rs</code> — Objek Sensor	9
4.4	File <code>controller.rs</code> — Pengambil Keputusan	10
4.5	File <code>monitoring.rs</code> — Pusat Kendali	11
5	Konsep OOP	11
5.1	Apa itu OOP dalam Rust?	11
5.2	Tiga Objek Utama Program	11
5.3	Keuntungan Menggunakan OOP	12
6	Implementasi Komputasi Numerik	12
6.1	Moving Average — Filter Data Sensor	12
6.1.1	Masalah yang Diselesaikan	12
6.1.2	Cara Kerja Moving Average	13
6.1.3	Implementasi dalam Rust	13
6.1.4	Contoh Nyata: Zona C Suhu Naik Bertahap	14
6.2	Kalibrasi Sensor	14
6.3	Statistik Deskriptif	14

7 Hasil Pengujian	15
7.1 Skenario Pengujian	15
7.2 Hasil Skenario 1 — Semua Normal	15
7.3 Hasil Skenario 3 — Danger Zona C	16
7.4 Hasil Skenario 5 — Moving Average Multi-Sesi	16
8 Analisis Hasil	16
8.1 Moving Average Mencegah Alarm Palsu	16
8.2 Alarm dan Kontrol Berjalan Sesuai Rencana	17
8.3 Deteksi Error Sensor Berhasil	17
8.4 Kelebihan dan Kekurangan Sistem	17
9 Kesimpulan	18

Bab 1. Pendahuluan

1.1 Latar Belakang

Gudang penyimpanan (*storage*) di industri farmasi harus selalu dijaga suhunya agar produk seperti obat dan vaksin tidak rusak. Jika suhu terlalu panas atau terlalu dingin, produk bisa gagal berfungsi dan berbahaya bagi konsumen.

Standar internasional menetapkan bahwa obat-obatan tertentu harus disimpan pada suhu antara 2°C sampai 8°C. Karena itu, dibutuhkan sistem yang bisa memantau suhu secara otomatis, memberi peringatan jika suhu menyimpang, dan mengaktifkan pendingin atau pemanas secara otomatis.

Pada proyek ini, kami membuat program monitoring suhu berbasis terminal menggunakan bahasa pemrograman Rust. Program ini dilengkapi dengan empat sensor suhu (satu per zona gudang), sistem alarm otomatis, dan filter data menggunakan *Moving Average*.

1.2 Rumusan Masalah

1. Bagaimana cara membuat program yang bisa memantau suhu di empat zona gudang sekaligus?
2. Bagaimana cara mendeteksi jika suhu terlalu tinggi atau terlalu rendah dan langsung memberi peringatan?
3. Bagaimana cara menyaring data sensor yang tidak stabil menggunakan metode *Moving Average*?
4. Bagaimana cara menggunakan konsep OOP (struct dan impl) di bahasa Rust untuk membangun sistem ini?

1.3 Tujuan

1. Membuat program monitoring suhu storage industri menggunakan Rust yang berjalan di terminal.
2. Menerapkan konsep OOP dengan tiga struct utama: **Sensor**, **Controller**, dan **MonitoringSystem**.
3. Menggunakan *Moving Average* sebagai metode numerik untuk menstabilkan data sensor.
4. Membangun sistem alarm otomatis dengan tiga tingkat: Normal, Warning, dan Danger.

1.4 Manfaat

Program ini bermanfaat untuk memahami cara kerja sistem instrumentasi nyata di industri, sekaligus melatih kemampuan pemrograman Rust, OOP, dan komputasi numerik secara terpadu.

Bab 2. Studi Kasus Instrumentasi

2.1 Deskripsi Sistem

Studi kasus yang dipilih adalah pemantauan suhu di gudang *cold room* milik PT. Industri Indonesia Hebat. Gudang ini menyimpan obat-obatan yang membutuhkan suhu dingin dan stabil. Gudang dibagi menjadi empat zona (Zona A, B, C, D), dan masing-masing zona dipasang satu sensor suhu.

Program akan membaca nilai suhu dari setiap sensor, menghitung rata-rata bergerak (*Moving Average*), lalu menentukan apakah kondisi gudang aman atau tidak.

2.2 Spesifikasi Sistem

Tabel 1: Spesifikasi Sistem Monitoring

Keterangan	Nilai	Penjelasan
Jumlah Zona	4 zona	Zona A, B, C, D
Jumlah Sensor	4 unit	1 sensor tiap zona
Suhu Normal	2,0–8,0°C	Standar obat-obatan
Batas Warning	0,0–2,0 dan 8,0–10,0°C	Zona waspada
Batas Danger	< 0,0 atau > 10,0°C	Zona bahaya
Range Sensor	–40,0 s.d. 85,0°C	Kemampuan alat
Window MA	5 data	Jumlah data untuk MA

2.3 Kategori Status Suhu

Program mengelompokkan kondisi suhu menjadi enam status:

Tabel 2: Kategori Status dan Tindakan Otomatis

No	Status	Range Suhu	Tindakan Program
1	NORMAL	$2,0 \leq T \leq 8,0^{\circ}\text{C}$	Tidak ada tindakan
2	WARNING TINGGI	$8,0 < T \leq 10,0^{\circ}\text{C}$	Tampilkan peringatan
3	WARNING RENDAH	$0,0 \leq T < 2,0^{\circ}\text{C}$	Tampilkan peringatan
4	DANGER PANAS	$T > 10,0^{\circ}\text{C}$	Alarm + pendingin ON
5	DANGER DINGIN	$T < 0,0^{\circ}\text{C}$	Alarm + pemanas ON
6	SENSOR ERROR	$T < -40$ atau $T > 85^{\circ}\text{C}$	Notifikasi error sensor

2.4 Hubungan dengan Teknik Instrumentasi

Sistem ini mencakup tiga proses utama dalam instrumentasi:

- **Pembacaan (Sensing):** Sensor membaca nilai suhu di tiap zona gudang.
- **Pengolahan Sinyal:** *Moving Average* menyaring nilai sensor agar lebih stabil.
- **Kontrol:** Program secara otomatis menyalakan pendingin atau pemanas jika suhu melewati batas bahaya.

Bab 3. Algoritma dan Flowchart

3.1 Algoritma Sistem

Algoritma adalah urutan langkah-langkah logis yang dijalankan program dari awal hingga selesai. Berikut algoritma sistem monitoring suhu yang dibuat:

Algoritma Monitoring Suhu Storage Industri

1. **MULAI**
2. Siapkan sistem: masukkan nama gudang, batas suhu, dan daftarkan 4 sensor (Zona A, B, C, D)
3. Tampilkan menu pilihan kepada pengguna
4. **Ulangi** selama pengguna belum memilih keluar:
 - a. Baca pilihan menu dari pengguna
 - b. Pilihan 1 $\rightarrow$ input suhu manual tiap sensor
 - c. Pilihan 2 $\rightarrow$ jalankan simulasi otomatis

- d. Pilihan 3 → tampilkan laporan lengkap
- e. Pilihan 4 → tampilkan demo Moving Average
- f. Pilihan 5 → tampilkan demo kalibrasi sensor

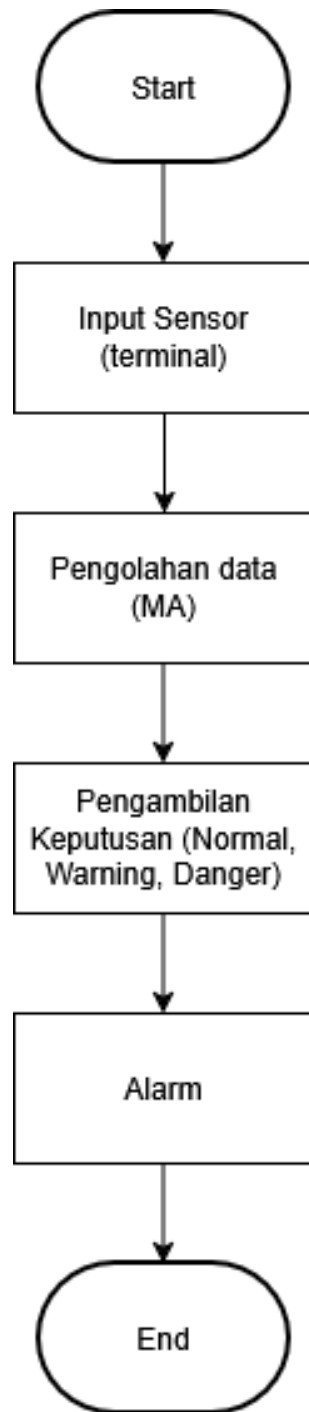
5. Proses tiap sesi monitoring:

- a. Baca nilai suhu dari setiap sensor
- b. Cek apakah nilai dalam batas kemampuan sensor
- c. Simpan nilai ke riwayat (history) sensor
- d. Hitung *Moving Average* dari riwayat
- e. Bandingkan nilai MA dengan batas suhu
- f. Tentukan status: Normal / Warning / Danger / Error
- g. Jika Warning atau Danger → tampilkan alarm
- h. Jika Danger Panas → aktifkan pendingin
- i. Jika Danger Dingin → aktifkan pemanas
- j. Tampilkan tabel hasil semua sensor
- k. Hitung statistik: rata-rata, min, max

6. SELESAI

3.2 Flowchart Sistem

Flowchart di bawah ini menggambarkan alur kerja program secara visual, mulai dari inisialisasi sistem hingga proses monitoring dan pengambilan keputusan alarm.



Gambar 1: Flowchart Sistem Monitoring Suhu Storage Industri

Bab 4. Penjelasan Program Rust

4.1 Struktur Program

Program dibagi menjadi lima file (modul) agar lebih rapi dan mudah dipahami. Setiap file punya tugas masing-masing:

Tabel 3: Daftar File Program dan Fungsinya

Nama File	Berisi	Tugasnya
main.rs	Fungsi utama	Menjalankan menu dan alur program
sensor.rs	Struct Sensor	Menyimpan data dan membaca suhu
controller.rs	Struct Controller	Menentukan status dan membunyikan alarm
monitoring.rs	Struct MonitoringSystem	Menggabungkan semua komponen
numerik.rs	Fungsi numerik	Menghitung Moving Average dan statistik

4.2 File main.rs — Pintu Masuk Program

File main.rs adalah bagian pertama yang dijalankan saat program dibuka. Di sini dibuat objek controller dan sistem monitoring, lalu sensor-sensor didaftarkan. Program kemudian masuk ke loop menu utama yang terus berjalan sampai pengguna memilih keluar.

```

1 fn main() {
2     // Buat controller dengan batas suhu cold room farmasi
3     let ctrl = Controller::new(
4         "Cold Room Storage PT. Industri Farma",
5         2.0,    // batas bawah suhu normal
6         8.0,    // batas atas suhu normal
7         2.0,    // jarak sebelum masuk zona bahaya
8     );
9
10    // Buat sistem monitoring, window MA = 5
11    let mut sistem = MonitoringSystem::new(
12        "PT. Industri Farma Nusantara", ctrl, 5
13    );
14
15    // Daftarkan 4 sensor, satu per zona
16    sistem.add_sensor(Sensor::new(1, "Sensor Zona A-01",
17                                    "Zona A", -40.0, 85.0));
18    sistem.add_sensor(Sensor::new(2, "Sensor Zona B-01",
19                                    "Zona B", -40.0, 85.0));
19

```

```

20     sistem.add_sensor(Sensor::new(3, "Sensor Zona C-01",
21                                   "Zona C", -40.0, 85.0));
22     sistem.add_sensor(Sensor::new(4, "Sensor Zona D-01",
23                                   "Zona D", -40.0, 85.0));
24     // Jalankan loop menu utama
25     loop {
26         tampilkan_menu();
27         let pilihan = baca_input_string("Masukkan pilihan: ");
28         match pilihan.trim() {
29             "1" => menu_monitoring_manual(&mut sistem),
30             "2" => menu_simulasi_otomatis(&mut sistem),
31             "0" => break, // keluar dari program
32             _   => println!("Pilihan tidak valid."),
33         }
34     }
35 }

```

Listing 1: Cara program dimulai di main.rs

Program menggunakan dua fitur penting Rust:

- `match`: seperti `if-else` berantai, digunakan untuk mencocokkan pilihan menu.
- `loop`: perulangan tanpa batas, program terus berjalan sampai pengguna memilih keluar (`break`).

4.3 File `sensor.rs` — Objek Sensor

File ini mendefinisikan seperti apa bentuk sebuah sensor dalam program. Sensor menyimpan nama, zona, nilai suhu terkini, dan riwayat pembacaan untuk keperluan *Moving Average*.

```

1  pub struct Sensor {
2      pub id: u32,           // nomor sensor
3      pub name: String,     // nama sensor
4      pub zone: String,     // zona gudang
5      pub value: f64,       // nilai suhu sekarang
6      pub min_range: f64,   // batas bawah kemampuan alat
7      pub max_range: f64,   // batas atas kemampuan alat
8      pub is_active: bool,  // apakah sensor aktif?
9      pub history: Vec<f64>, // riwayat nilai untuk MA
10 }
11

```

```

12 impl Sensor {
13     // Simpan nilai baru dan tambahkan ke riwayat
14     pub fn set_value(&mut self, val: f64) {
15         self.value = val;
16         self.history.push(val);
17         // Hanya simpan 10 data terakhir
18         if self.history.len() > 10 {
19             self.history.remove(0);
20         }
21     }
22
23     // Cek apakah nilai sensor masuk akal (tidak rusak)
24     pub fn is_valid(&self) -> bool {
25         self.value >= self.min_range &&
26         self.value <= self.max_range
27     }
28 }

```

Listing 2: Definisi struct Sensor

4.4 File controller.rs — Pengambil Keputusan

File ini berisi logika untuk menentukan apakah suhu dalam kondisi aman atau tidak. Ada enam kemungkinan status yang bisa dihasilkan.

```

1 // Daftar kemungkinan status suhu
2 pub enum StatusSuhu {
3     Normal, WarningTinggi, WarningRendah,
4     DangerTinggi, DangerRendah, SensorError,
5 }
6
7 impl Controller {
8     // Fungsi utama: tentukan status berdasarkan nilai MA
9     pub fn check_status(&self, suhu: f64,
10                         sensor_valid: bool) -> StatusSuhu {
11         // Jika sensor rusak, langsung error
12         if !sensor_valid {
13             return StatusSuhu::SensorError;
14         }
15         // Bandingkan suhu dengan batas-batas yang sudah
16         // diatur
17         if suhu > self.normal_max + self.warning_dev {

```

```
17         StatusSuhu::DangerTinggi    // terlalu panas
18     } else if suhu > self.normal_max {
19         StatusSuhu::WarningTinggi    // mendekati panas
20     } else if suhu < self.normal_min - self.warning_dev {
21         StatusSuhu::DangerRendah    // terlalu dingin
22     } else if suhu < self.normal_min {
23         StatusSuhu::WarningRendah    // mendekati dingin
24     } else {
25         StatusSuhu::Normal            // aman
26     }
27 }
28 }
```

Listing 3: Enum status dan logika pengecekan suhu

4.5 File `monitoring.rs` — Pusat Kendali

File ini adalah penghubung semua bagian program. Struct `MonitoringSystem` menyimpan daftar sensor dan controller, lalu menjalankan satu sesi monitoring lengkap: baca data → hitung MA → cek status → tampilkan hasil.

Bab 5. Konsep OOP

5.1 Apa itu OOP dalam Rust?

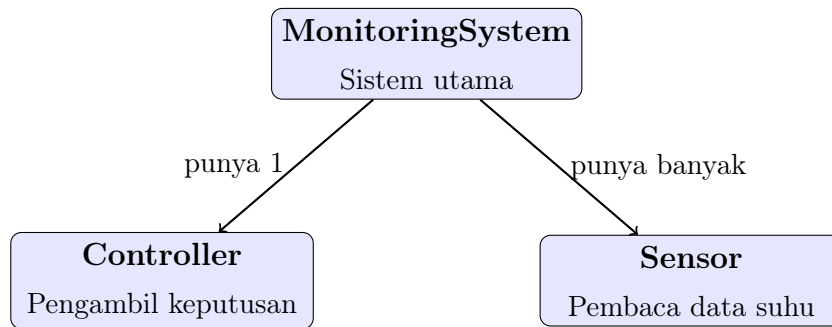
OOP (*Object-Oriented Programming*) adalah cara membuat program dengan membungkus data dan fungsi-fungsinya ke dalam satu kesatuan yang disebut **objek**.

Di Rust, objek dibuat menggunakan dua kata kunci:

- **struct**: mendefinisikan *atribut* (data yang disimpan objek). Contoh: sensor punya `name`, `value`, `zone`.
- **impl**: mendefinisikan *method* (fungsi yang bisa dilakukan objek). Contoh: sensor bisa `set_value()`, `is_valid()`, `display()`.

5.2 Tiga Objek Utama Program

Program ini memiliki tiga struct utama yang saling bekerja sama:



Gambar 2: Hubungan Antar Objek dalam Program

Tabel 4: Penjelasan Tiga Objek Utama

Nama Objek	Menyimpan Apa	Bisa Melakukan Apa
Sensor	ID, nama, zona, nilai suhu, riwayat data	Simpan nilai baru, cek apakah sensor valid
Controller	Batas suhu normal, status pendingin/pemanas	Tentukan status suhu, bunyikan alarm, nyalakan aktuator
MonitoringSystem	Daftar sensor, controller, log data	Jalankan sesi monitoring, tampilkan laporan, hitung statistik

5.3 Keuntungan Menggunakan OOP

- **Rapi:** Setiap bagian program punya tanggung jawab sendiri, tidak campur aduk.
- **Mudah dikembangkan:** Ingin tambah sensor? Cukup panggil `add_sensor()` tanpa mengubah kode lain.
- **Mudah dipahami:** Nama struct dan method mencerminkan fungsinya di dunia nyata.

Bab 6. Implementasi Komputasi Numerik

6.1 Moving Average — Filter Data Sensor

6.1.1 Masalah yang Diselesaikan

Sensor suhu di dunia nyata tidak selalu membaca nilai yang stabil. Meskipun suhu ruangan tetap 5°C, sensor bisa membaca 4,9 lalu 5,2 lalu 4,8 karena ada gangguan listrik kecil (disebut *noise*).

Jika program langsung menggunakan nilai mentah ini untuk memutuskan alarm, bisa terjadi alarm palsu. Solusinya adalah **Moving Average**: ambil rata-rata dari beberapa data terakhir, sehingga nilai menjadi lebih stabil.

6.1.2 Cara Kerja Moving Average

Window size = 5 artinya kita selalu menghitung rata-rata dari **5 nilai terakhir**. Setiap ada data baru, data paling lama dibuang.

$$MA = \frac{x_1 + x_2 + x_3 + x_4 + x_5}{5} \quad (1)$$

Contoh sederhana dengan window = 3:

Tabel 5: Contoh Sederhana Cara Kerja Moving Average

Data Baru	3 Data Terakhir	MA	Keterangan
4,1	[4,1]	4,10	Baru 1 data
4,5	[4,1; 4,5]	4,30	Baru 2 data
3,9	[4,1; 4,5; 3,9]	4,17	Window penuh
5,2	[4,5; 3,9; 5,2]	4,53	Data pertama dibuang
4,8	[3,9; 5,2; 4,8]	4,63	Terus bergeser

6.1.3 Implementasi dalam Rust

```

1 pub fn moving_average(data: &[f64], window: usize) -> Vec<f64>
2 {
3
4     let mut hasil = Vec::new();
5
6     for i in 0..data.len() {
7         if i + 1 < window {
8             // Belum cukup data, rata-rata dari yang ada
9             let jumlah: f64 = data[..i].iter().sum();
10            hasil.push(jumlah / (i + 1) as f64);
11        } else {
12            // Sudah cukup data, ambil 'window' data terakhir
13            let jumlah: f64 = data[(i+1-window)..=i].iter().
14                sum();
15            hasil.push(jumlah / window as f64);
16        }
17    }
18    hasil

```

16 }

Listing 4: Fungsi Moving Average di numerik.rs

6.1.4 Contoh Nyata: Zona C Suhu Naik Bertahap

Berikut hasil MA untuk Zona C saat simulasi multi-sesi, menggunakan window = 5:

Tabel 6: Moving Average Zona C — Simulasi 5 Sesi

Sesi	Suhu Raw	5 Data Terakhir	MA	Status
1	4,50°C	[4,50]	4,50°C	Normal
2	5,80°C	[4,50; 5,80]	5,15°C	Normal
3	7,20°C	[4,50; 5,80; 7,20]	5,83°C	Normal
4	9,10°C	[4,50; 5,80; 7,20; 9,10]	6,65°C	Normal
5	11,50°C	[4,50; 5,80; 7,20; 9,10; 11,50]	7,62°C	Normal

Pada sesi ke-5, suhu mentah sudah 11,50°C (seharusnya Danger), tetapi MA masih 7,62°C sehingga status tetap Normal. Ini karena MA memperhitungkan data-data sebelumnya yang masih rendah.

Poin penting: Program menggunakan nilai MA (bukan suhu mentah) sebagai acuan pengambilan keputusan. Tujuannya agar alarm tidak berbunyi karena lonjakan sesaat yang bukan kondisi nyata berbahaya.

6.2 Kalibrasi Sensor

Kalibrasi adalah proses mengoreksi nilai sensor agar lebih mendekati nilai sebenarnya. Rumusnya sangat sederhana:

$$T_{\text{terkoreksi}} = (T_{\text{sensor}} \times \text{gain}) + \text{offset} \quad (2)$$

- **Gain:** pengali untuk mengoreksi skala (biasanya mendekati 1,0).
- **Offset:** nilai tambah/kurang untuk mengoreksi geseran pembacaan.

Contoh: sensor membaca 10,0°C, gain = 1,02, offset = −0,2 maka nilai terkoreksi = $(10,0 \times 1,02) + (-0,2) = 10,0^\circ\text{C}$.

6.3 Statistik Deskriptif

Program juga menghitung tiga statistik sederhana setiap akhir sesi untuk memberikan gambaran kondisi keseluruhan:

- **Rata-rata** semua zona: $\bar{x} = \frac{\text{jumlah semua nilai}}{\text{banyak data}}$
- **Nilai tertinggi** (maksimum) antar zona
- **Nilai terendah** (minimum) antar zona

Bab 7. Hasil Pengujian

7.1 Skenario Pengujian

Program diuji dengan lima skenario untuk memastikan semua fitur berjalan dengan benar:

Tabel 7: Daftar Skenario Pengujian

Skenario	Kondisi yang Diuji	Yang Diharapkan
1	Semua zona normal	Tidak ada alarm, status semua NORMAL
2	Zona B mendekati batas atas	Muncul alarm WARNING
3	Zona C suhu sangat tinggi	Alarm DANGER + pendingin menyala
4	Sensor Zona D nilai error	Muncul notifikasi SENSOR ERROR
5	Suhu naik 5 sesi bertahap	MA bergerak lambat, alarm telat

7.2 Hasil Skenario 1 — Semua Normal

Tabel 8: Hasil Skenario 1: Semua Zona Normal

ID	Sensor	Zona	Suhu	MA	Status
1	Sensor Zona A-01	Zona A	4,20°C	4,20°C	NORMAL
2	Sensor Zona B-01	Zona B	5,10°C	5,10°C	NORMAL
3	Sensor Zona C-01	Zona C	3,80°C	3,80°C	NORMAL
4	Sensor Zona D-01	Zona D	6,00°C	6,00°C	NORMAL
Rata-rata sistem			4,78°C	—	—

7.3 Hasil Skenario 3 — Danger Zona C

Tabel 9: Hasil Skenario 3: Zona C Terlalu Panas

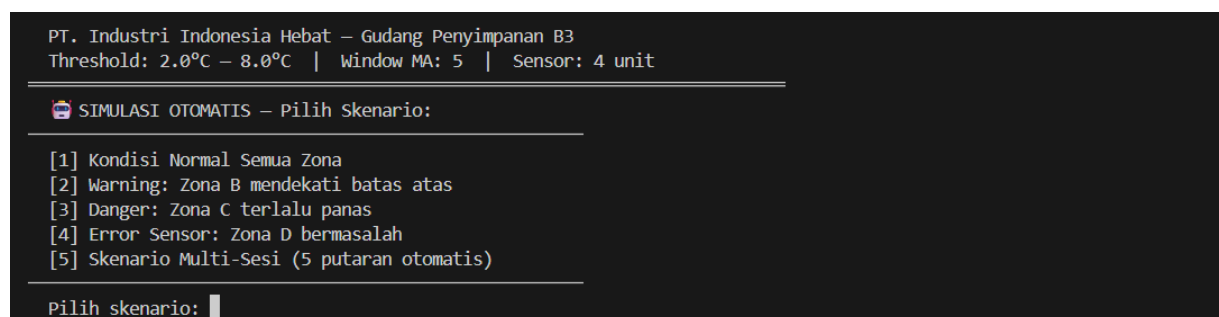
ID	Sensor	Zona	Suhu	MA	Status
1	Sensor Zona A-01	Zona A	4,10°C	4,10°C	NORMAL
2	Sensor Zona B-01	Zona B	5,00°C	5,00°C	NORMAL
3	Sensor Zona C-01	Zona C	12,50°C	12,50°C	DANGER PANAS
4	Sensor Zona D-01	Zona D	3,90°C	3,90°C	NORMAL

⇒ Alarm DANGER aktif. Pendingin Zona C dinyalakan otomatis oleh program.

7.4 Hasil Skenario 5 — Moving Average Multi-Sesi

Tabel 10: Pengaruh Moving Average — Zona C Suhu Naik Bertahap

Sesi	Suhu Mentah	MA	Status
1	4,50°C	4,50°C	NORMAL
2	5,80°C	5,15°C	NORMAL
3	7,20°C	5,83°C	NORMAL
4	9,10°C	6,65°C	NORMAL
5	11,50°C	7,62°C	NORMAL



Gambar 3: Screenshot Tampilan Program Saat Dijalankan

Bab 8. Analisis Hasil

8.1 Moving Average Mencegah Alarm Palsu

Dari hasil Skenario 5 terlihat bahwa walaupun suhu mentah Zona C sudah mencapai 11,50°C pada sesi ke-5, nilai MA masih 7,62°C dan status tetap Normal.

Ini terjadi karena MA menghitung rata-rata dari lima sesi terakhir yang masih banyak berisi nilai rendah. Artinya, satu lonjakan suhu sesaat tidak langsung memicu alarm. Kondisi ini sesuai dengan cara kerja sistem instrumentasi industri nyata, di mana alarm hanya berbunyi jika kondisi berbahaya sudah berlangsung cukup lama, bukan sekadar lonjakan sesaat.

8.2 Alarm dan Kontrol Berjalan Sesuai Rencana

Pada Skenario 3, saat MA Zona C mencapai $12,50^{\circ}\text{C}$ (melewati batas Danger $10,0^{\circ}\text{C}$), program langsung menampilkan pesan alarm merah dan mengaktifkan pendingin secara otomatis. Ini membuktikan bahwa logika kontrol di **Controller** bekerja dengan benar.

Pada Skenario 2, saat suhu masuk zona Warning, program hanya menampilkan peringatan tanpa menyalakan aktuator — sesuai desain bahwa Warning adalah notifikasi awal, bukan kondisi darurat.

8.3 Deteksi Error Sensor Berhasil

Pada Skenario 4, nilai sensor yang dimasukkan (200°C) jauh melampaui kemampuan alat (85°C). Program berhasil mendeteksinya sebagai Sensor Error, bukan sebagai kondisi suhu berbahaya. Ini penting karena dalam industri nyata, sensor yang rusak harus dibedakan dari suhu yang memang sedang tinggi.

8.4 Kelebihan dan Kekurangan Sistem

Tabel 11: Evaluasi Kelebihan dan Kekurangan

Kelebihan	Kekurangan
Filter MA mencegah alarm palsu	Data sensor masih diinput manual
Alarm otomatis 3 tingkat (Normal/Warning/Danger)	Belum tersambung ke sensor fisik nyata
Deteksi sensor error secara otomatis	Data tidak tersimpan ke file setelah program ditutup
Kontrol pendingin dan pemanas otomatis	Tampilan masih berbasis teks, belum ada grafik
Struktur modular, mudah dikembangkan	Belum ada fitur ekspor laporan

Bab 9. Kesimpulan

Dari proses perancangan, pembuatan, dan pengujian program, dapat disimpulkan hal-hal berikut:

1. Program monitoring suhu storage industri berhasil dibuat menggunakan bahasa Rust dan berjalan di terminal dengan baik.
2. Konsep OOP berhasil diterapkan menggunakan tiga struct: **Sensor** (membaca data suhu), **Controller** (menentukan status dan alarm), dan **MonitoringSystem** (mengintegrasikan seluruh sistem).
3. Metode *Moving Average* dengan window 5 berhasil menstabilkan data sensor. Nilai MA digunakan sebagai acuan keputusan sehingga alarm tidak mudah berbunyi karena lonjakan sesaat.
4. Sistem alarm tiga tingkat (Normal, Warning, Danger) bekerja sesuai rancangan. Alarm Danger secara otomatis mengaktifkan pendingin atau pemanas.
5. Fitur deteksi error sensor berhasil membedakan antara suhu yang memang tinggi dan sensor yang rusak berdasarkan batas kemampuan alat.
6. Program ini dapat dikembangkan lebih lanjut dengan menambahkan koneksi ke sensor fisik, penyimpanan data ke file, dan tampilan grafis berbasis GUI.

Pustaka

- [1] Klabnik, S. & Nichols, C. (2023). *The Rust Programming Language*, 2nd ed. No Starch Press. <https://doc.rust-lang.org/book/>
- [2] World Health Organization. (2014). *WHO Good Manufacturing Practices for Pharmaceutical Products*. WHO Technical Report Series No. 986.
- [3] Burden, R.L. & Faires, J.D. (2015). *Numerical Analysis*, 10th ed. Cengage Learning.
- [4] Morris, A.S. (2001). *Measurement and Instrumentation Principles*. Butterworth-Heinemann.
- [5] Smith, S.W. (1997). *The Scientist and Engineer's Guide to Digital Signal Processing*. California Technical Publishing. <http://www.dspguide.com/>